

Distributed Systems – Labs

Python - Programming with HTTP for the Internet

Tasks

Task 1. Downloading data from an HTTP server

We will see how to write a simple HTTP client to fetch some data from any web server using the native HTTP protocol. This can be the very first steps towards creating your own HTTP browser.

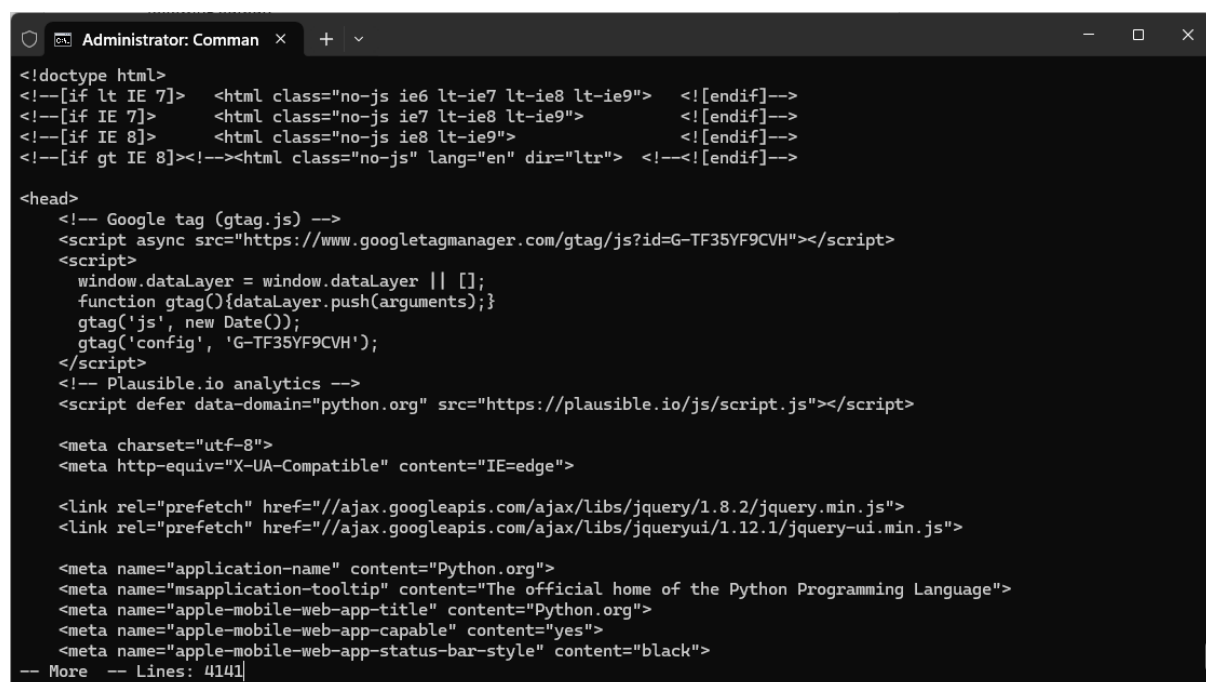
Examine `PT1\4_1_download_data.py` that is a code for a simple HTTP client:

`4_1_download_data.py` will by default fetch a page from <https://www.w3schools.com/>.

You can run this recipe with or without the host argument. You may choose to fetch any specific web page by passing the URL as an argument. If this script is executed, it will show the following output:

```
> python 4_1_download_data.py --host=http://www.python.org
```

You will get output similar to this:



```
<!doctype html>
<!--[if lt IE 7]> <html class="no-js ie6 lt-ie7 lt-ie8 lt-ie9"> <![endif]-->
<!--[if IE 7]> <html class="no-js ie7 lt-ie8 lt-ie9"> <![endif]-->
<!--[if IE 8]> <html class="no-js ie8 lt-ie9"> <![endif]-->
<!--[if gt IE 8]><!--><html class="no-js" lang="en" dir="ltr"> <!--<![endif]-->

<head>
  <!-- Google tag (gtag.js) -->
  <script async src="https://www.googletagmanager.com/gtag/js?id=G-TF35YF9CVH"></script>
  <script>
    window.dataLayer = window.dataLayer || [];
    function gtag(){dataLayer.push(arguments);}
    gtag('js', new Date());
    gtag('config', 'G-TF35YF9CVH');
  </script>
  <!-- Plausible.io analytics -->
  <script defer data-domain="python.org" src="https://plausible.io/js/script.js"></script>

  <meta charset="utf-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">

  <link rel="prefetch" href="//ajax.googleapis.com/ajax/libs/jquery/1.8.2/jquery.min.js">
  <link rel="prefetch" href="//ajax.googleapis.com/ajax/libs/jqueryui/1.12.1/jquery-ui.min.js">

  <meta name="application-name" content="Python.org">
  <meta name="msapplication-tooltip" content="The official home of the Python Programming Language">
  <meta name="apple-mobile-web-app-title" content="Python.org">
  <meta name="apple-mobile-web-app-capable" content="yes">
  <meta name="apple-mobile-web-app-status-bar-style" content="black">
-- More -- Lines: 4141
```

This recipe will also work for any page in the sites. Not just the home page:

```
> python 4_1_download_data.py --host=https://www.python.org/downloads/
```

This recipe defines an `urllib.request` module that fetches data from the remote host. `urllib.request.urlopen()` opens the given web page and fetches it.

Task 2: Serving HTTP requests from your machine

We will see how to create your own web server. Your web server should handle client requests and send a simple hello message.

Python ships with a very simple web server that can be launched from the command line as follows:

```
➤ python -m SimpleHTTPServer 8080
```

As we are running Python 3, you will get error as ‘No module named SimpleHTTPServer’. It’s because in python 3, SimpleHTTPServer has been merged into `http.server` module. You can use below command to run python http server in Python 3.

`http.server` is not recommended for production. It only implements basic security checks. In Python 3 you start it as follows, passing port number to run.

```
➤ python -m http.server 5000
```

Or

```
➤ python3 -m http.server 5000
```

This will launch an HTTP web server on port 5000. You can access this web server from your browser by typing `http://localhost:5000` or `http://127.0.0.1:5000`

This will show the contents of the current directory from where you run the preceding command. If there is any web server index file, for example, `index.html`, inside that directory, your browser will show the contents of `index.html`.

If you want to create your own web server, you need to launch your customized HTTP server.

Examine the code in `PT1\4_2_simple_http_server.py`

`4_2_simple_http_server.py` gives the code for the simple custom HTTP web server:

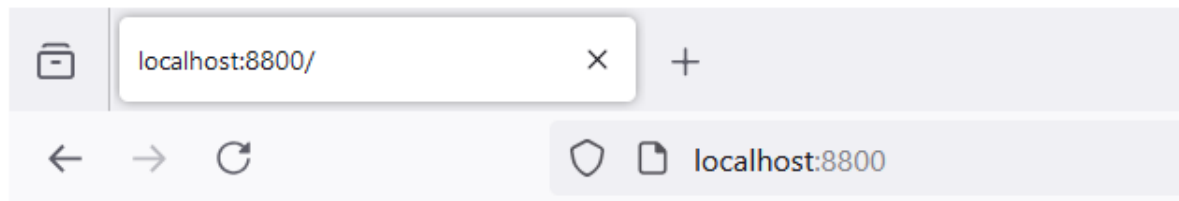
When you run it you should get something similar to this|:

```
>python 4_2_simple_http_server.py --port=8800
```

Custom HTTP server started on port: 8800

127.0.0.1 - - [07/Oct/2024 12:06:34] "GET / HTTP/1.1" 200 -

If you run this web server and access the URL from a browser (`http://localhost:8800`), this will send the one text 'Hello from server!' to the browser, like this:



Hello from server!

In this recipe (`4_2_simple_http_server.py`), created `CustomHTTPServer` class inherited from the `HTTPServer` class. In the constructor method, the `CustomHTTPServer` class sets up the server address and port received as a user input. In the constructor, our web server's `RequestHandler` class has been set up. Every time a client is connected, the server handles the request according to this class.

The `RequestHandler` defines the action to handle the client's GET request. It sends an HTTP header (code 200) with a success message 'Hello from server!' using the `write()` method.

References

- Chapter 4, Kathiravelu P., Sarker F., Python Network Programming Cookbook, Second Edition, Packt Publishing Ltd, 2017
- <https://www.python.org/>